

Design and Implementation of a 2D Side-Scrolling Shooter Game Using Python and Pygame

[Ankur Bandopadhyay]

Department of Computer Science | [Kalinga Institute of Industrial Technology(KIIT)]

[Kolkata, West Bengal, India] | [23053105@kiit.ac.in]

Article Info

Keywords: Pygame, Python, Game Development, OOP, Tile Map, AI, Scrolling Shooter

Project Reference: YouTube Tutorial Series — Pygame Shooter (12 Parts)

Tools: Python 3.x, Pygame 2.x, CSV, OS module

Platform: Windows / Linux / macOS

Abstract

A soldier shows up on screen, moving through levels built from tiles loaded via CSV files. Though made with help from a dozen video guides, the code took its own path early on. One class handles both enemies and the main character, swapping roles when needed. Tiles form the world, while separate systems manage how things move - bullets fly, grenades arc downward. Animation runs frame by frame, tied to movement direction and speed. Each level unfolds step by step, triggered by progress, not timers. Code loops constantly, checking inputs, updating positions, redrawing everything every fraction of a second. Behind it all, classes split tasks: one tracks health, another manages collisions, a third pushes objects down like gravity does. Projectiles follow paths shaped by rules mimicking real motion. Enemy behavior shifts based on proximity, not complex decision trees. Design stays clear, parts connect without tangles. Structure appears in layers - one for visuals, one for logic, one for flow. Details matter most where action happens: landing shots, avoiding fire, stepping forward after each defeat. A working game with several layers of interaction comes through clearly in the findings. This effort acts like a starting point for learning how to build games using objects in Python, showing basic code ideas turned into something you can play. Though built simply, it reveals how logic becomes movement on screen. Each piece connects - structure meets behavior - without extra tools or complex frameworks. Learning happens by doing here, where small steps add up to real results. Code takes shape not just as theory but as actions unfolding

1. INTRODUCTION

Creating playable video games stands among the broadest tests of real-world computing skills. Alongside coding structure, it blends motion math, screen drawing, response systems, plus logic design - all at once. Thanks to clear rules in writing code and strong outside tools, one language - Python - has grown into a go-to choice for building

these experiences. Especially when using the toolkit called Pygame, what seems complex becomes manageable without losing strength.

A fresh take on crafting a 2D side-scroller emerges through Python, powered by Pygame. Instead of starting from zero, it pulls ideas from a dozen-video guide shaped by someone who teaches game building step by step. Each piece came together slowly - worlds made of tiles that shift as you move, enemies that act on their own - not just copied but adjusted, tested, fixed until they worked. What stands now is a complete run-through experience, level after level, built block by block without skipping steps.

One main goal stood out early: learning how events trigger actions in live programs. Working directly with moving characters taught timing through trial, then error. Shapes bouncing across the screen revealed hidden flaws in thinking. Each crash between objects needed careful tracking, not guesswork. Patterns emerged when organizing pieces into reusable chunks. Building it step by step forced attention to detail others might skip. Tweaking parts beyond the guide showed where theory broke down. Fixing broken logic became routine after the third failed run. Custom changes proved understanding better than any test could.

After this introduction comes a look at Pygame and how tile-based games are built. Next up, an overview of how the whole system fits together unfolds gradually. What follows dives into how every part was actually put into code. Later, what worked - and where things got tricky - comes under review. Finally, some closing thoughts appear, along with possible paths ahead.

2. BACKGROUND AND RELATED CONCEPTS

2.1 Python and Pygame

A favorite in classrooms and labs, Python powers everything from beginner lessons to complex number crunching. Thanks to clear syntax and ready-to-use tools, picking up coding feels less like climbing a wall. Built for crafting games without headaches, Pygame brings together several Python pieces under one roof. Instead of wrestling pixels and sound drivers directly, developers tap into pre-built features that just work. Underneath, it leans on SDL - a lean engine handling raw media tasks so creators can focus on gameplay. Smooth access to screen drawing, user actions, timing, and sound turns ideas into interactive experiences fast.

2.2 Tile-Based Game Design

Tile-based games represent a foundational design paradigm in 2D game development. In this model, the game world is divided into a grid of uniform square tiles, each representing a specific type of terrain, obstacle, or interactive element. This approach simplifies level design, collision detection, and rendering. The game developed in this project uses a 16-row by 150-column tile grid, with 21 distinct tile types, stored as integer values in CSV files. This allows for efficient level design, easy modification, and programmatic loading of levels.

2.3 Object-Oriented Design in Games

Games fit well with object-oriented programming. Not every piece holds the same role - player, foe, shot, thrown explosive - but each acts like a separate thing, carrying traits and actions inside. Built-in tools in Pygame, like Sprite and Group, already support this way of building things. A single Soldier class appears again and again, taking different forms depending on setup at creation time, whether human-controlled or opponent. Other pieces join it: bullets fly, grenades burst into flame, items drop boxes, health gets measured visually, terrain builds up block by block - all tied under one system that reflects how parts belong together.

2.4 Reference Tutorial Series

The project is based upon a twelve-part YouTube tutorial series covering the development of a complete Pygame shooter game from scratch. The series progressively introduces concepts including display setup, character sprites, animation systems, physics, collision detection, tile maps, enemy AI, weapons, HUD elements, pickups, and

multi-level progression. This paper draws on that series as the primary pedagogical reference, with the author's own code constituting the final implemented artefact.

3. SYSTEM DESIGN AND ARCHITECTURE

3.1 Game Configuration Constants

The game is configured through a set of global constants defined at program initialization. These constants establish the dimensions and physical properties of the game environment:

```
SCREEN_WIDTH = 500
SCREEN_HEIGHT = int(SCREEN_WIDTH * 0.8) # 400 px - 80% of width
GRAVITY = 0.75 # pixels/frame²
SCROLL_THRESH = 200 # px before screen scrolls
ROWS = 16 # tile rows in the world grid
TILE_SIZE = SCREEN_HEIGHT // ROWS # 25 px per tile
COLS = 150 # total tile columns per level
TILE_TYPES = 21 # distinct tile image types
MAX_LEVELS = 3 # number of playable levels
FPS = 60 # frame rate cap
```

The tile size is derived programmatically from the screen height and row count, ensuring that the tile grid always fills the screen exactly regardless of resolution adjustments. This design choice promotes scalability.

3.2 Class Architecture

The game is structured around eight primary classes, each inheriting from `pygame.sprite.Sprite` where applicable. The table below summarises the class hierarchy and responsibilities:

Class	Role	Key Methods
Soldier	Player & Enemy characters	<code>move()</code> , <code>shoot()</code> , <code>ai()</code> , <code>update_animation()</code>
World	Level tile map loader	<code>process_data()</code> , <code>draw()</code>
Bullet	Bullet projectile	<code>update()</code> — movement & collision
Grenade	Thrown explosive	<code>update()</code> — physics, timer, blast
Explosion	Visual blast effect	<code>update()</code> — frame animation
Itembox	Pickup items on map	<code>update()</code> — collision & pickup
HealthBar	HUD health display	<code>draw()</code> — ratio-based rendering
Decoration/Water/Exit	Environment tiles	<code>update()</code> — scroll sync

All sprite classes register themselves into Pygame Sprite Groups, enabling batch updates and batch rendering through a single `draw()` call. This architectural choice significantly simplifies the main game loop.

3.3 Game Loop Structure

Each cycle runs like clockwork, hitting 60 steps every second. Starting fresh each time, it checks timing first to keep pace steady. Then comes drawing - backgrounds stretch out before worlds unfold beneath them. Characters shift position after that; players move alongside foes, bullets fly through bursts of sparks. Overlaid on top, status markers appear - hearts dwindle while clips refill and grenades wait ready. Finally, signals flow in - a key press here, a click there, screens changing when needed. By splitting duties this way, actions stay clear without tangled visuals.

4. IMPLEMENTATION

4.1 Game Initialization and Window Setup

Out of nowhere, Pygame kicks things off - setting up the screen right after. Right then, every picture needed gets pulled into memory just one time. Because loading later would slow everything down, they sit ready from the start. Each tile type points straight to its image through a numbered list. That means pulling a tile is fast - almost instant, really. Main menu buttons? Same deal, brought in now alongside the rest. Later on, when drawing the world, nothing waits for files anymore. Frame timing begins here too, ticking steadily beneath it all.

```
pygame.init()
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption('Shooter')
clock = pygame.time.Clock()

# Pre-load all 21 tile images into indexed list
img_list = []
for x in range(TILE_TYPES):
    img = pygame.image.load(f'img/Tile/{x}.png')
    img = pygame.transform.scale(img, (TILE_SIZE, TILE_SIZE))
    img_list.append(img)
```

4.2 The Soldier Class — Player and Enemy

A soldier stands at the heart of the game's structure. This blueprint builds every playable figure along with hostile figures, shaped through a label - either 'player' or 'enemy'. When brought to life, it pulls together animations: resting, moving forward, lifting off ground, ending. Each comes alive from folders filled with picture steps loaded one after another. How many images appear in each motion? That count rises or falls based on what sits inside those directories, counted fresh each time via `os.listdir()`. Flexibility lives here, built quietly into how things unfold behind sight.

A character stays active when `alive` shows true. Movement pace depends on speed value set. Bullets ready to fire come from ammo count. Grenades are tracked separately as a number. Health points go down when hit. Direction flips left or right using plus one or minus one. Upward or downward motion lives inside `vel_y`. Touching floor means `in_air` turns false. Each move has its own sequence in `animation_list`, built as layers within lists.

```
class Soldier(pygame.sprite.Sprite):
    def __init__(self, char_type, x, y, scale, speed, ammo, grenades):
        pygame.sprite.Sprite.__init__(self)
        self.alive = True
        self.char_type = char_type
        self.speed = speed
        self.ammo = ammo
        self.grenades = grenades
        self.health = 100
        self.direction = 1 # 1 = right, -1 = left
        self.vel_y = 0
        self.in_air = True
        self.action = 0 # 0=Idle, 1=Run, 2=Jump, 3=Death
        # AI-specific
        self.vision = pygame.Rect(0, 0, 150, 20)
        self.idling = False
        self.move_counter = 0
```

4.3 Movement, Physics, and Jumping

When keys press left or right, the `move()` function gets those signals as true or false values. Depending on direction, `dx` becomes either positive or negative speed. The character turns its look around through `pygame.transform.flip()`. Facing shifts based on where it's going across screen.

Every frame, gravity adds 0.75 pixels per second squared to downward speed. Upward motion begins when jump happens - speed becomes exactly minus eleven. Midair movement keeps building unless stopped by ground contact. Landing on solid tiles zeroes out vertical momentum. Once feet touch floor, jumping ability returns. Motion stays locked until surface detection fires. Jumping twice mid-flight never works because air status blocks it.

```
def move(self, moving_left, moving_right):
    dx, dy = 0, 0
    if moving_left: dx = -self.speed; self.flip = True
    if moving_right: dx = self.speed; self.flip = False
    if self.jump and not self.in_air:
        self.vel_y = -11 # launch upward
        self.jump = False
        self.in_air = True
    self.vel_y += GRAVITY # apply gravity each frame
    dy += self.vel_y
```

Screen scrolling is also managed within the move() method. When the player approaches within SCROLL_THRESH (200 px) of either screen edge, the player's position is held fixed while a screen_scroll offset is returned to the caller, which then shifts all world objects and background layers accordingly.

4.4 Collision Detection

Bumping into stuff gets checked using the obstacle_list stored inside the World class - this list holds every solid tile, numbered from 0 to 8. Each tile faces a box-shaped check, one direction at a time: first horizontal, then vertical. Splitting it like that keeps the player from getting trapped on sharp edges, something that often happens if both directions get handled together. Hitting your head shows up when val_y points upward during a vertical clash; when that happens, val_y drops to zero. Touching down after a fall triggers once val_y stops going downward.

Hitting water ends things fast - health drops to zero without delay. When a character touches the exit tile, level_complete switches to True right away. Falling past the bottom of the screen counts as failure too. Bullets meeting characters trigger responses through spritecollide(). Group collisions rely on that function to detect contact accurately.

4.5 Enemy Artificial Intelligence

How enemies act comes from the ai() function, using a basic setup with just three modes: walking around, standing still, or striking. Moving forward until they cover one full tile's length defines their walk - then they turn and go the opposite way. Every single frame while roaming, each foe rolls a tiny dice; if it lands right (just once every two hundred times), motion halts abruptly. That pause lasts fifty steps on screen, silent and frozen, before wandering starts again.

A small viewing area, sized 150 by 20 pixels, sits centered on the enemy and stretches 75 pixels forward based on where it's facing. This viewing zone checks overlap with the player's space every update. Once contact happens, movement stops immediately - then a shot launches. As the enemy shifts, so does this detection window, keeping aim aligned with its path.

```
def ai(self):
    if self.alive and player.alive:
        if self.vision.collidect(player.rect): # player detected
            self.update_action(0) # stop running
            self.shoot() # open fire
        else:
            if not self.idling:
                self.move(ai_moving_left, ai_moving_right)
                self.move_counter += 1
                if self.move_counter > TILE_SIZE:
                    self.direction *= -1 # reverse direction
                    self.move_counter *= -1
```

4.6 Weapon Systems — Bullets and Grenades

A single shot moves sideways every time the frame updates - ten steps each tick, aimed where the shooter faced. Motion keeps up with the shifting background thanks to `screen_scroll` adjustments tucked into the `update()` cycle. When one vanishes beyond view, smacks a wall, or strikes a figure - it ends right there. If launched by the player, it drains thirty health points from enemies hit. Foes fire back with weaker shots: only five damage dealt when they land.

A small explosive flies through the air when tossed, moving like real objects would. It begins with a forward push at 7 pixels every frame, while shooting upward at a rate of 11. Each update, gravity pulls it down just a bit more. Hitting a wall flips its travel direction sideways, though reaching a ceiling kills that sideways motion entirely. Once launched, a clock starts ticking - only 100 steps before it blows apart. When time runs out, a burst appears right where it lands, covering everything nearby within two tile lengths. Anything caught in that circle loses half its health instantly.

4.7 Explosion Visual Effects

The `Explosion` class renders a sequence of five explosion frames at the detonation point. A sudden burst shows five image stages right where it blows up. Every tick, a number inside goes up, switching pictures based on `EXPLOSION_SPEED` - four ticks per change. When the last picture appears, the object vanishes completely, taken out of every group via `kill()`, cleaning up space without help. Just like the soldier moves through images, this too steps forward one by one.

4.8 Tile Map World System

The `World` class handles how the tile map gets read and drawn. CSV files hold level details - `level1_data.csv` is one example - with every spot on the grid holding a number for a tile kind. Inside that 16 by 150 setup, `process_data()` checks each entry; whenever it finds a number zero or more, it brings in the matching piece of gameplay. `Obstacle_list` collects blocks when numbers fall between 0 and 8. Water shows up if the digit hits 9 or 10. Decorative bits appear for values from 11 through 14. Player begins at whatever space holds exactly 15. Enemies pop onto the scene wherever there's a 16. Item boxes form when digits land on 17, 18, or 19. Exit lands right where the number reaches 20.

```
with open(f'level{level}_data.csv', newline='') as csvfile:
    reader = csv.reader(csvfile, delimiter=',')
    for x, row in enumerate(reader):
        for y, tile in enumerate(row):
            world_data[x][y] = int(tile)

world = World()
player, health_bar = world.process_data(world_data)
```

Level completion is detected by collision between the player and the Exit sprite. Upon completion, the level counter increments, all sprite groups are cleared via `reset_level()`, and the next CSV is loaded. If the maximum level is exceeded, the game ends.

4.9 Item Boxes and Health Bar (HUD)

One kind of box gives health, fixed at max if already full, each refill worth 25 HP. When touching the player, checked via Pygame's rectangle overlap tool, it vanishes while boosting the right stat. Another type adds fifteen bullets, appearing only once per pickup. A third holds three grenades, gone on contact, increasing explosive count instantly. Overlapping shapes form the health display - black frame first, then red base showing total possible strength. Green layer shrinks or grows based on how much health exists now compared to limit. Width matches ratio exactly, drawn fresh every update. Each item behaves separately but shares removal logic after touch.

4.10 Parallax Scrolling Background

A sense of depth begins when background pieces shift at separate speeds while the screen moves. Instead of one flat image, four parts slide separately - clouds up high, then mountains, distant pines, and close-up trees. Their motion adjusts through values: half-speed, slightly more, even slower increases down the line. Things meant to seem nearer zip by quicker than those meant to feel far away. Across the horizontal span, every piece repeats itself exactly five times so gaps never show during movement.

```
def draw_bg():
    screen.fill(BG)    # base green fill
    width = sky_img.get_width()
    for x in range(5):
        screen.blit(sky_img,      ((x*width) - bg_scroll*0.5, 0))
        screen.blit(mountain_img, ((x*width) - bg_scroll*0.6, ...))
        screen.blit(pine1_img,    ((x*width) - bg_scroll*0.7, ...))
        screen.blit(pine2_img,    ((x*width) - bg_scroll*0.8, ...))
```

4.11 Main Menu and Button System

One Button class - brought in from button.py - takes care of clickable interface parts like Start, Exit, and Restart. Its image and location live inside each button instance. When it's time to show one, the draw() function puts that image on screen, checking at once whether the mouse floats over or clicks it. A switch called start_game controls if the game runs normally or waits at the menu during the central loop. After a death occurs, up pops the restart option, reloading just the stage instead of launching everything again..

4.12 Multi-Level System and Level Progression

Level progression is managed by the global level integer, which begins at 1 and increments upon each level completion. The system supports up to MAX_LEVELS (3) levels, each defined by its own CSV tile map. The reset_level() function empties all sprite groups and returns a fresh empty tile data array. The level data is then re-read from the appropriate CSV file and a new World object is constructed. This modular approach allows new levels to be added simply by creating additional CSV files, without modifying any game logic code.

5. RESULTS AND DISCUSSION

Finished version works completely as a 2D side-moving shooting game across several stages. Moving through grid-like worlds comes naturally, while enemies act on their own, requiring attention. Health refills, bullet supplies, grenades appear along the path - each collected during play. Three unique areas unfold step by step without skipping ahead. Runs steadily aiming for sixty frames each second on regular computers. Scrolling feels fluid, reactions to inputs stay sharp throughout. Performance holds firm where it should.

5.1 Functional Features Achieved

- Smooth horizontal scrolling with camera threshold and parallax depth layers
- Fully animated player and enemy sprites with Idle, Run, Jump, and Death states
- Physics-accurate jumping and gravity, with solid tile collision detection
- Autonomous enemy AI with patrol, idle, and attack states, and directional vision sensing
- Dual weapon system: rapid-fire bullets and physics-driven grenades with blast radius
- Three types of collectible item boxes dynamically spawned from the tile map
- Dynamic HUD: real-time health bar, bullet count, and grenade count display
- Multi-level progression loaded from CSV tile maps with full state reset between levels
- Main menu with Start/Exit, and in-game Restart upon player death

5.2 Challenges and Solutions

A few tricky problems popped up while building things. Instead of getting caught on edges, sprites now slide free thanks to split-phase checks for side and top collisions. Jumping twice midair stopped being possible once the game started watching the `in_air` signal more closely. For the scrolling backdrop, tile lengths had to be fine-tuned so seams wouldn't show when moving across uneven distances.

Flipping directions confused enemy bots until someone added a `move_counter`. That number grows with every step taken, piling up distance like loose change. Once it passes a full tile's width, the system flips which way the bot faces. Throwing grenades used to flood the screen if the button stayed down. Now there is a `grenade_thrown` marker watching closely. It blocks new grenades from appearing mid-press. Only when the key lifts does the signal reset, allowing another launch later.

5.3 Limitations

Right now, it falls short in more than a few ways. Sound playback is missing entirely. Enemy behavior runs on basic rules instead of smart navigation. There's no score tracking, no countdown clock, nor saved records between sessions. When sprites have odd outlines, square hit-boxes can miss the mark. Room to improve shows up right there.

6. CONCLUSION

Right off, the work shows a full 2D side-scrolling shooter made in Python using Pygame. Layered on top comes the use of object-oriented methods, shaping small pieces into something that works together smoothly. Take the `Soldier` class - it handles both friend and foe without rewriting core logic. Instead of hardcoding maps, levels snap together via basic CSV files thanks to the grid-style World setup. Movement feels grounded because physics rules guide speed and bumps between objects. Enemies act less like scripts, more like threats, since their decisions shift mid-play.

A step-by-step twelve-part guide shaped how things came together, while solo troubleshooting and tweaks filled in the gaps - this mix opened a clear window into live event handling, moving sprites, loops that drive gameplay, plus keeping track of changing conditions. What stands now runs start to finish, spans several stages, can grow later.

Few tweaks down the line might include smarter enemy movement with something like A star pathfinding. Level creation could shift into the game itself through a built-in tile map editor. Sound bits and tunes may come alive via Pygame's mixer tools. Progress tracking might appear by adding saved games and score logging. Online play could creep in using sockets from Python's standard kit. Each addition nudges the project further away from classroom demo toward standalone app territory.

R. REFERENCES

- [1] T. Ninja, "Pygame Shooter Game — Part 1: Project Setup & Window," YouTube, 2021. [Online]. Available: <https://youtu.be/DHgj5jhMJKg>
- [2] T. Ninja, "Pygame Shooter Game — Part 2: Player Sprite & Movement," YouTube, 2021. [Online]. Available: <https://youtu.be/7WsE1Vy1Krg>
- [3] T. Ninja, "Pygame Shooter Game — Part 3: Sprite Animation," YouTube, 2021. [Online]. Available: <https://youtu.be/Su5OILSpJDU>
- [4] T. Ninja, "Pygame Shooter Game — Part 4: Scrolling World," YouTube, 2021. [Online]. Available: <https://youtu.be/fEQ8Cle23ko>

- [5] T. Ninja, "Pygame Shooter Game — Part 5: Tile Map & World," YouTube, 2021. [Online]. Available: <https://youtu.be/wqBCWdm9bGY>
- [6] T. Ninja, "Pygame Shooter Game — Part 6: Enemies & AI," YouTube, 2021. [Online]. Available: <https://youtu.be/cS9cagVNSGc>
- [7] T. Ninja, "Pygame Shooter Game — Part 7: Shooting & Bullets," YouTube, 2021. [Online]. Available: <https://youtu.be/IVer6kbsARY>
- [8] T. Ninja, "Pygame Shooter Game — Part 8: Grenades & Explosions," YouTube, 2021. [Online]. Available: <https://youtu.be/1WxQt4-ic3s>
- [9] T. Ninja, "Pygame Shooter Game — Part 9: Health Bar & HUD," YouTube, 2021. [Online]. Available: <https://youtu.be/5VSzZu5904c>
- [10] T. Ninja, "Pygame Shooter Game — Part 10: Item Pickups," YouTube, 2021. [Online]. Available: https://youtu.be/m7GnJo_oZUU
- [11] T. Ninja, "Pygame Shooter Game — Part 11: Level Design & CSV," YouTube, 2021. [Online]. Available: https://youtu.be/JTM8_pcQOUU
- [12] T. Ninja, "Pygame Shooter Game — Part 12: Game Menu & Completion," YouTube, 2021. [Online]. Available: <https://youtu.be/wWLDUb71mFo>
- [13] Pygame Community, "Pygame Documentation v2.x," 2024. [Online]. Available: <https://www.pygame.org/docs/>
- [14] Python Software Foundation, "Python 3 Standard Library — os, csv, random modules," 2024. [Online]. Available: <https://docs.python.org/3/library/>
- [15] R. Nystrom, Game Programming Patterns, Genever Benning, 2014. [Online]. Available: <https://gameprogrammingpatterns.com>